6 Functions

6.1 Introduction

When working with bigger programs, or programs performing many different tasks, it is important to divide the code into small limited pieces, which makes it easier to grasp and maintain. Functions are used for this purpose. A function is a part of the program well marked off that performs a particular task. A function can be reused several times in a program or in many different programs.

A function, when completed and tested, is like a black box that always works as expected. You don't need to bother any more about what is inside. You give it a number of input values, and it does its job.

Think for instance of a car driver who wants to increase the speed. He pushes the accelerator and the car accelerates. He doesn't have to bother about fuel injection, gear ratio, engine compression and the like. He knows that the same thing always happens when pushing the accelerator. It is the same with functions. The programmer initiates the function from his code and the function always performs the same task without having to bother about the details.

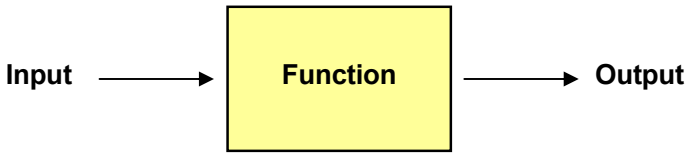
To summarize, the advantages with functions are:

- Small well marked off program sections
- Well-structured programs
- Easier to debug and maintain
- Reusage of code

In this chapter we will learn to write functions, supply input values (parameters) and receive the result from a function. We will learn how to declare and define functions and we will study how to use header files in connection with functions. We will also learn about reference parameters – an efficient tool to save memory and improve program performance. We will also get in touch with recursive functions, i.e. functions calling themselves.

6.2 What Is a Function

A function is written to perform a specific task. It might need input values and it returns the result from the task being performed:



The programmer to make the function perform its task must supply required input and receive the supplied output.

Examples of function tasks are:

- Calculate average – here you will have to supply the detailed values for the average calculation, and receive the average value delivered by the function.
- Sort an array – here the function must know which array to sort. It returns the sorted array.

Download free eBooks at bookboon.com

- Search for an item in an array – here the function also must know which array to search. It returns the item found.
- Calculate order price – here the input may consist of quantity, unit price and a discount factor. The output is the calculated order price.

6.3 Average

We will write a function which calculates the average of two numbers. The two numbers form the input, and the calculated average is the output.

The task is to add the two numbers and divide the sum by 2. Suppose that the two variables x1 and x2 contain the input values. The statement to calculate the average is then:

```
av = (x1 + x2) / 2;
```

The variable av now contains the calculated average.

The entire function is coded in this way:

```
double dAverage(double x1, double x2)
{
  double av;
  av = (x1 + x2)/2;
  return av;
}
```

The function has a name, dAverage. The required input values are enumerated within parenthesis after the function name. They are called **formal parameters** and are named x1 and x2. Furthermore, you specify the *data type* in question. Both x1 and x2 are of the double type. The data type is given in front of each parameter, and the parameters are separated by comma.

At the first line we also specify the data type for the *result value* in front of the function name. In our case the result is of the double type.

Thus, the first line specifies the name of the function, the required input and the output. The first line is called **function header**.

The task to be performed by the function is described by the code inside the curly brackets. This section is called the **function body** and consists of three statements. First we declare a variable av, which is required inside the function. At the second line we perform the average calculation. The average is stored in the variable av. At the third line we return that value to the caller. The statement

```
return av;
```

means that the value in the variable av is delivered as output. A return statement also has the effect that the function is terminated. The task has been completed.

6.4 Calling a Function

The function dAverage is complete, but it does not perform anything yet. Furthermore, the parameters x1 and x2 don't have any values, so the statement with the average calculation is meaningless so far. We must make the function start.

Download free eBooks at bookboon.com

We must *call* the function.

Calling a function means three things:

- Write the function name
- Supply input to the function
- Receive the return value

The following statement calls the function dAverage:

```
dAvg = dAverage(dNo1, dNo2);
```

This statement implies that the function dAverage is initiated and that the values contained by the variables dNo1 and dNo2 are supplied as input to the function. A prerequisite for this is of course that the variables dNo1 and dNo2 have been assigned values prior to the function call. The variables dNo1 and dNo1 are called **actual parameters**, since they contain actual values.

The execution is now passed over to the function. The function header:

```
double dAverage(double x1, double x2)
```

tells us that there are two formal parameters x1 and x2. The actual parameter values are copied to the formal parameters in the specified sequence. x1 will get the value of dNo1 and x2 the value of dNo2. Now that the formal parameters have got their values the statements of the function body are meaningful and can be executed.

In the function body the variable av is declared, the average is calculated and stored in av, which is returned. Then the function has completed.

TURN TO THE EXPERTS FOR
SUBSCRIPTION CONSULTANCY

Subscrybe is one of the leading companies in Europe when it comes to innovation and business development within subscription businesses.

We innovate new subscription business models or improve existing ones. We do business reviews of existing subscription businesses and we develope acquisition and retention strategies.

Learn more at linkedin.com/company/subscrybe or contact
Managing Director Morten Suhr Hansen at mha@subscrybe.dk

SUBSCRYBE - to the future

Click on the ad to read more

Download free eBooks at bookboon.com

The execution is now passed over to the calling statement:

```
dAvg = dAverage(dNo1, dNo2);
```

The entire right part is now replaced by the returned value, which is assigned to the variable dAvg. At completion of the statement the variable dAvg has got the value returned by the function, namely the average of dNo1 and dNo1.

Here is the entire program

```
#include <iostream.h>
double dAverage(double x1, double x2)
{
  double av;
  av = (x1 + x2)/2;
  return av;
}
void main()
{
  double dNo1, dNo2, dAvg;
  cout << "Enter two numbers: ";
  cin >> dNo1 >> dNo2;
  dAvg = dAverage(dNo1, dNo2);
  cout << "The average is " << dAvg;
}
```

The execution of a program always starts with the main() function, and here the variables dNo1, dNo2 and dAvg are declared. Then the user is prompted for two numbers to be stored in the variables dNo1 and dNo2. Then comes the statement:

```
dAvg = dAverage(dNo1, dNo2);
```

which calls the function dAverage and supplies the two entered values. The function does its job and returns the average, which is stored in the variable dAvg. The last statement prints the calculated average.

You may ask why we don't write the main() function first, since it is the starting point of the execution. The reason is that the compiler must know about the function dAverage before it is called from the main() function. Therefore the compiler must first process dAverage. We will discuss this more later in this chapter.

6.5 Several return Statements

A function might need to return different values depending on the circumstances. Consequently, a function can contain several return statements. However, it is always only one return statement performed by the function at execution. As soon as a return statement is executed, the function completes. Here is an example of a function with two return statements:

```
int min(int x, int y)
{
```


Download free eBooks at bookboon.com

```
   if (x<y)
      return x;
   else
      return y;
}
```

The function returns the least of two integers. By examining the function header (the first line) we figure out that the function takes two integers stored in the formal parameters x and y and that it returns an integer.

The function body contains an if statement that checks whether x is less than y. If so, x is returned, otherwise y is returned. This implies that the function always returns the least of the two numbers.

6.6 Least of Three Numbers

We will now use the function min() in a program that reads three integers from the keyboard and prints the least of them.

Since the function min() only can compare two numbers, we will have to call it twice. We compare the two first integers entered by the user and get as a result the least of these two. That integer is then compared to the third number, which again gives the least of them. As a result we get the least of all three integers. Here is the code:

```
int a, b, c, m;
cout << "Enter three integers: ";
cin >> a >> b >> c;
m = min(a,b);
m = min(m,c);
cout << m;
```

We declare the three variables a, b and c for storing of the input values, and a variable m used for storing of the value returned from the function min().

The function min() is called with a and b as actual parameters. The least of these two is returned and stored in the variable m. The function min() is again called with m and c as actual parameters. The least of these two is returned and stored in the variable m, which now contains the least of the three integers. Finally that value is printed.

Here is the entire program:

```
#include <iostream.h>
int min(int x, int y)
{
   if (x<y)
      return x;
   else
      return y;
}
void main()
{
```


Download free eBooks at bookboon.com

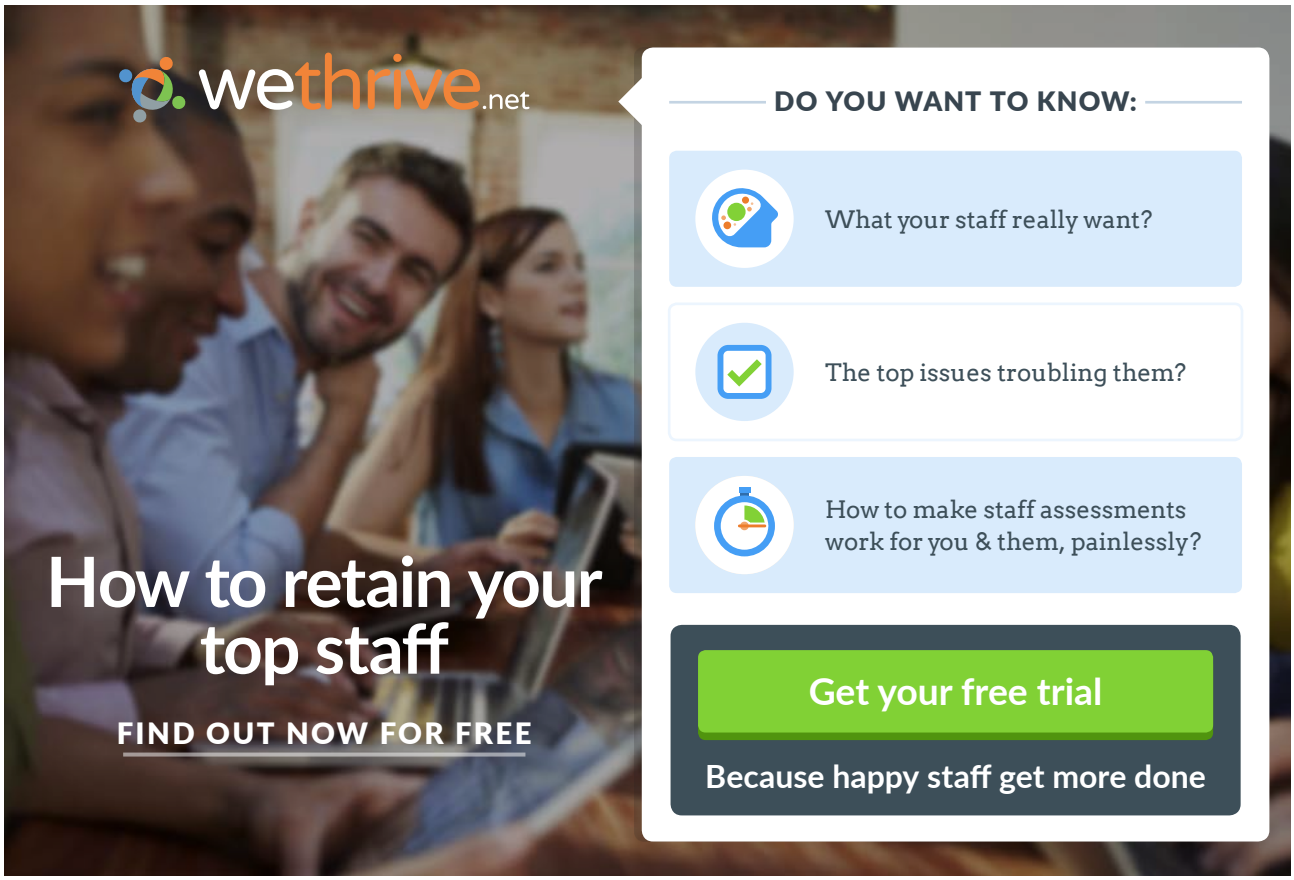
```
   int a, b, c, m;
   cout << "Enter three integers: ";
   cin >> a >> b >> c;
   m = min(a,b);
   m = min(m,c);
   cout << m;
}
```

As mentioned, the execution starts in main(). The reason for placing the function min() before main() is for the compiler to recognize it when calling it from main().

An alternative to calling the function min() in two different statements is given here:

```
cout << min(min(a,b),c) << " is the least";
```

This statement replaces the three last statements of the preceding program. First the program tries to execute the outer min() call. But since the first actual parameter is not an ordinary variable value, the program must calculate it, and is then forced to execute the inner min() call. As a result it returns the least of a and b, which now can be used as actual parameter to the outer min(). The second actual parameter to the outer min() call is the variable c. The outer min() returns the least of the three integers, which is printed by the cout statement.



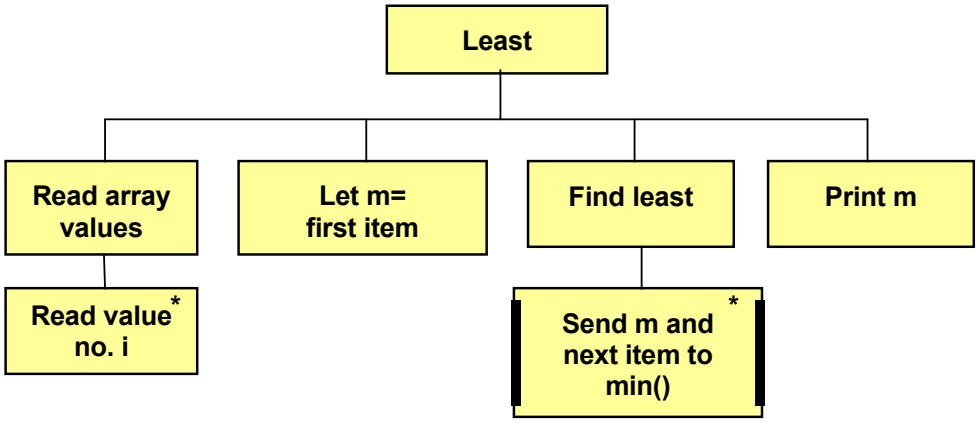
Click on the ad to read more

Download free eBooks at bookboon.com

6.7 Least Item of an Array

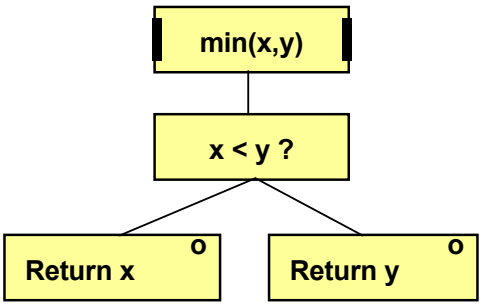
We will now use the min() function in a program that finds the least of the items of an integer array. The logic is similar to that of the previous program. The first two items of the array are sent to min(), which returns the least of these two integers. This integer and the next item of the array is again sent to min() which returns the least of them. This process is repeated until all items of the array have been processed. As a result we will get the least of the array items.

First we give a JSP graph:



Entry of values is made in a loop. Then we set the variable m equal to the first item of the array. In the loop 'Find least' we send m and the next item to the function min(). The return value is stored in the variable m. Since we use m to store the returned value, m will all the time contain the least of the items compared so far. At completion of the loop we print the m value, which now is the least of all array items.

We have used thicker border lines to the box with the function call to indicate that it is a function. We create a separate JSP graph for the function:



As shown by the JSP the function takes two parameters, x and y, and checks whether x is less than y. If so, x is returned, otherwise y.

Here is the program code:

```
#include <iostream.h>
int min(int x, int y)
{
```


Download free eBooks at bookboon.com

```
   if (x<y)
      return x;
   else
      return y;
}
void main()
{
   int iNos[5], i, m;
   for (i=0; i<=4; i++)
   {
      cout << "Enter integer no. " << i+1 << ": ";
      cin >> iNos[i];
   }
   m=iNos[0];
   for (i=1; i<=4; i++)
   {
      m=min(iNos[i],m);
   }
   cout << m << " is the least integer" << endl;
}
```

In main() we declare the array iNos with 5 items. The first for-loop reads the integers from the user to the array. Then we assign the first array item to the variable m.

The second for-loop calls the function min() repeatedly with m and next array item as actual parameters. The returned value is stored in m. Finally we print the value of m.

6.8 Array As Parameter

Sometimes you want to send an entire array as parameter to a function, especially when working with strings. The function header for such a function could look like this:

```
int iLgth(char s[])
```

The function iLgth takes a parameter of char type. The formal parameter name is s. The empty square bracket indicates that it is an array. Note that the square bracket does not contain any value indicating the number of items of the array. The reason is that the function should be able to manage arrays of any size, so we don't want to lock the function to any fixed array size.

The function header also shows that the function returns a value of integer type, namely the length of the string array. The calculation of the length is made by the function body:

```
int iLgth(char s[])
{
   int n=0;
```


Download free eBooks at bookboon.com

```
   while (s[n] != '\0')
      n++;
   return n;
}
```

First a variable n is declared initialized to 0, which is to count the number of characters in the string s. The while loop has the condition that the nth character must not be the null character, i.e. it proceeds until the end of the string. When the nth character equals the null character, the string end has been reached and n then contains the number of characters in the string, which is returned.

When the function iLgth is called with an array as actual parameter, you don't specify any square brackets:

```
iLen = iLgth(cWord);
```

Here, cWord is a string array that contains a number of characters. The returned length of the string is stored in the variable iLen.

Below is a program that tests the function iLgth:

```
#include <iostream.h>
int iLgth(char s[])
{
   int n=0;
   while (s[n] != '\0')
      n++;
```


Download free eBooks at bookboon.com

```
   return n;
}
void main()
{
   char cWord[30];
       int iLen;
       cout << "Enter a word: ";
   cin.getline(cWord,29);
   iLen = iLgth(cWord);
   cout << "The length of the word is " << iLen << endl;
}
```

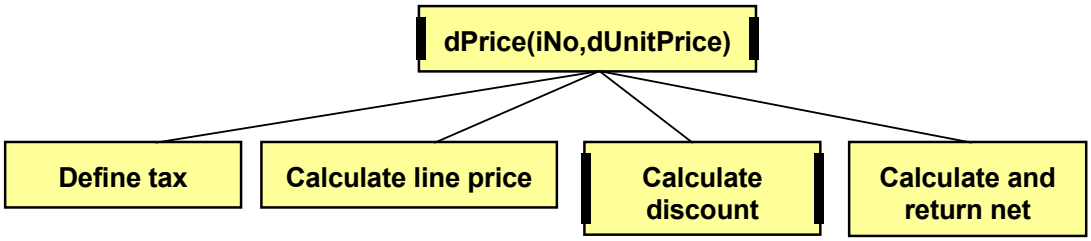
In main() we declare the string array cWord with max 30 characters, and the integer variable iLen, which later will contain the string length. A text is read from the keyboard. Then the function iLgth() is called with the entered string as actual parameter. The function returns the length of the string which is stored in the variable iLen and printed.

6.9 Function and Subfunction

It is possible to write functions to be called by other functions providing a hierarchy of functions and subfunctions. This is rather common in bigger programs and provides well-structured programs, where each function performs a limited and well defined task.

We will now write a function, dPrice(), which calculates the price of a product and also calls another function, dDiscount(), which calculates and returns a discount percent. The percent is then used by dPrice() to calculate the discounted price.

First we write a JSP graph for the function dPrice():



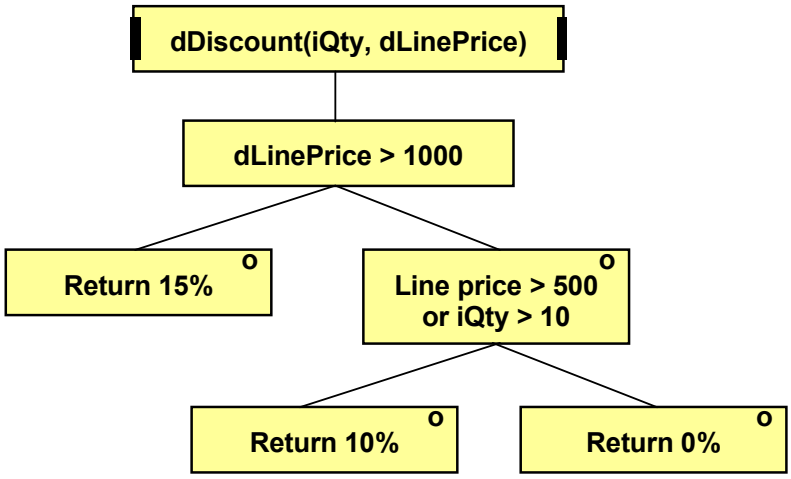
The function takes two parameters, number of units and the unit price.

The function defines the tax factor to 0.25. The line price is calculated as number of units times unit price. The discount is then calculated by sending the quantity and line price. Later, the discount will be dependent of both the quantity and line price. The function dDiscount returns a discount percent to be used for calculation of the discounted net price, which is returned by dPrice().

There are some thicker side lines indicating functions.

The JSP graph for the dDiscount() function looks like this:

Download free eBooks at bookboon.com



The function dDiscount() takes the quantity and line price as parameters. If the line price exceeds 1000, the discount 15 % is returned. Otherwise (i.e. the line price is less than or equal to 1000), we check if the line price exceeds 500 or the quantity exceeds 10. Then 10 % is returned. In other cases there will be no discount and 0 % is returned.

We begin the coding with the dDiscount() function. When writing programs with functions it is convenient to start with the functions at the bottom of the hierarchy.

```
double dDiscount(int iQty, double dLinePrice)
{
  if (dLinePrice>1000)
    return 0.15;
  else if (dLinePrice>500 || iQty>10)
    return 0.10;
  else
    return 0;
}
```

The function dDiscount() takes the quantity as parameter, which is stored in the formal parameter iQty, and the line price stored in the parameter dLinePrice. The if statement checks the values and returns a percentage of the double type.

Below is the code for the dPrice() function:

```
double dPrice(int iNo, double dUnitPrice)
{
  const double dTax = 0.25;
  double dLinePr, dDiscPerc;
  dLinePr = iNo * dUnitPrice;
  dDiscPerc = dDiscount(iNo, dLinePr);
  return dLinePr * (1-dDiscPerc)*(1+dTax);
}
```

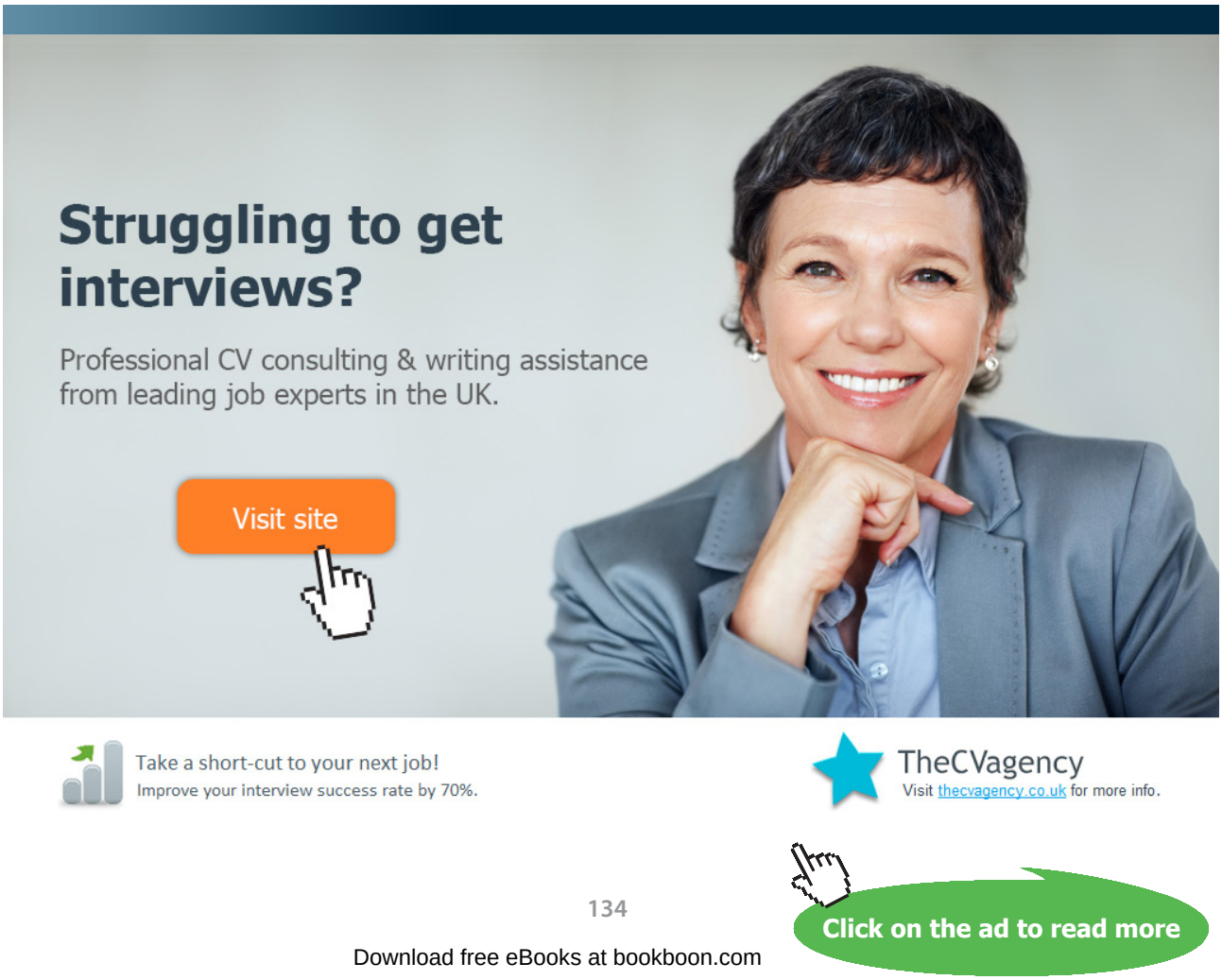

Download free eBooks at bookboon.com

The function dPrice() takes the quantity and unit price as parameters. First a constant named dTax is declared. Then variables for the line price and discount percent are declared. The line price is calculated as quantity times unit price.

Then the function dDiscount() is called with quantity and line price as actual parameters. The returned discount percentage is stored in the variable dDiscPerc. Finally, the discounted net price of double type is returned, where we deduct the discount percentage and add the tax percentage.

Here is an entire program that tests the functions:

```
#include <iostream.h>
double dDiscount(int iQty, double dLinePrice)
{
   if (dLinePrice>1000)
      return 0.15;
   else if (dLinePrice>500 || iQty>10)
      return 0.10;
   else
      return 0;
}
double dPrice(int iNo, double dUnitPrice)
{
   const double dTax = 0.25;
   double dLinePr, dDiscPerc;
```



Download free eBooks at bookboon.com

```
   dLinePr = iNo * dUnitPrice;
   dDiscPerc = dDiscount(iNo, dLinePr);
   return dLinePr * (1-dDiscPerc)*(1+dTax);
}
void main()
{
      int iQuantity;
      double dUnitPrice;
      cout << "Enter quantity and unit price: ";
   cin >> iQuantity >> dUnitPrice;
   cout << "To be paid: "
         << dPrice(iQuantity, dUnitPrice)<<endl;
}
```

In main() we declare the variables iQuantity and dUnitPrice, which store the entered values. The cout statement calls the dPrice() function with the entered values as actual parameters. As return value we get the discounted net price, which is printed by the cout statement.

6.10 Function without Return Value

Some functions are supposed to perform a task but don't need to return any value. An example could be a function that prints a message or performs some string manipulation.

We will create a function that prints the character '=' a specific number of times. It can be used to print a number of equality signs acting as underlining a text. We use the name underline for the function:

```
void underline(int n)
{
   for (int i=1; i<=n; i++)
     cout << "=";
}
```

A function that does not return any value has 'void' in front of the function name. Compare the function main() which does not have to return any value to the operating system, and which consequently has been defined as 'void main()'.

The function underline() takes an integer as parameter. It has a for-loop that prints the character '=' as many times as given by the integer.

We can now use this function in a program to underline a text:

```
char s[7] = "Prices";
cout << s << endl;
underline(strlen(s));
```

This code section defines a string array s with the string 'Prices'. The string is printed at the second code line and then we call the function underline() with the length of the string as actual parameter. This gives the output:

Download free eBooks at bookboon.com

```
Priser
======
```

An alternative to the function underline is given here:

```
void underline(char text[])
{
   for (int i=1; i<=strlen(text); i++)
      cout << "=";
}
```

Here, the function instead takes the string itself as parameter. The for-loop goes from 1 to the number of characters in the string and prints as many equality signs. The call to this variant of the function should be:

```
char s[7] = "Prices";
cout << s << endl;
underline(s);
```

Here we send the string variable as actual parameter instead of the length of the string.

6.11 Replacing Characters in a String

We will create still another void function that replaces an arbitrary character in a string and prints the modified string. We call it replace:

```
void replace(char s[], char c, char cnew)
{
   int n = 0;
   while (s[n] != '\0')
   {
      if (s[n] == c)
         s[n] = cnew;
      n++;
   }
   cout << s << endl;
}
```

The function takes three parameters, a string array - s, the character to be replaced – c, and the replacing character – cnew.

The function first defines a variable n to be used for indication of one character at a time in the string. The while loop has the condition that the character must not be the null character, i.e. we proceed from the first to the last character of the string.

The if statement checks whether the character in the string equals the character to be replaced (c). If so, that character iw replaced by cny. In the while loop we finally increase n by 1 to point out the next character in the string.

Finally the modified string is printed by the cout statement.

Download free eBooks at bookboon.com

Here is an entire program that tests the replace() function:

```
#include <iostream.h>
void replace(char s[], char c, char cnew)
{
   int n= 0;
   while (s[n] != '\0')
   {
      if (s[n] == c)
         s[n] = cnew;
      n++;
   }
   cout << s << endl;
}
void main()
{
   char a[100] = "C:/Mydocuments/Sheets";
   cout << a << endl;
   replace(a, '/', '-');
}
```

Click on the ad to read more

Download free eBooks at bookboon.com